PREPPILOT

Interview Trainer Agent

TESTING REPORT

Evidence screenshots for Knowledge Base Retrieval, Answer Evaluation, and Incorrect Answer Evaluation.

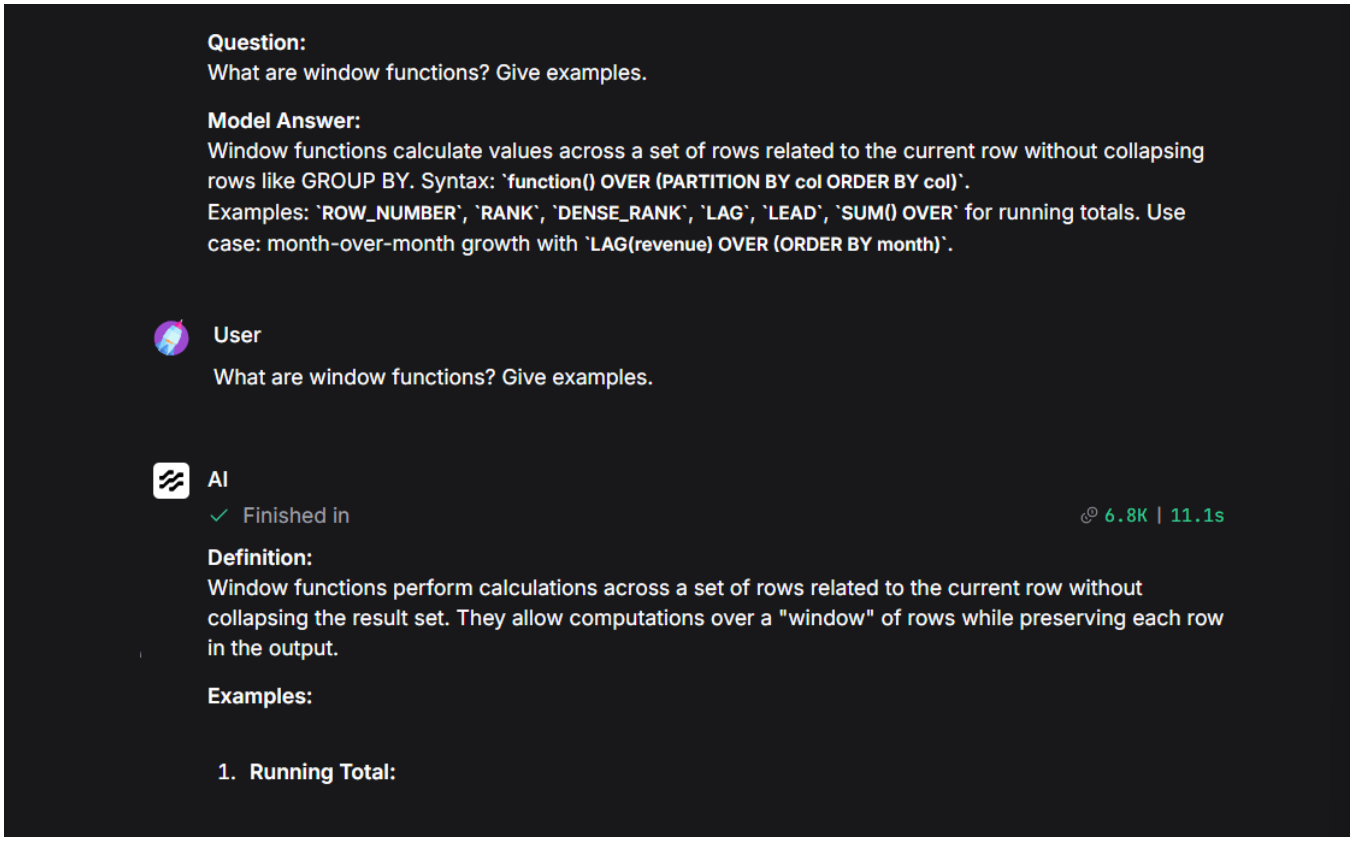
Project: PrepPilot — Interview Trainer Agent for Data Analyst / Data Scientist practice

Purpose: Document the three observed Playground tests using the screenshots provided.

Integrity note: Scores and responses are represented from the supplied screenshots; the original test outputs have not been changed.

Test 1 — Knowledge Base Retrieval

Observed result: the Knowledge Base returned a relevant model answer for “What are window functions? Give examples.” The retrieved content includes a definition, syntax, common examples, and a month-over-month growth use case.



Screenshot 1 — Test 1 — Knowledge Base Retrieval

1. **Running Total:**

sql

```sql
SELECT
    order_date,
    amount,
    SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM orders;
```

1. **Rank within Department:**

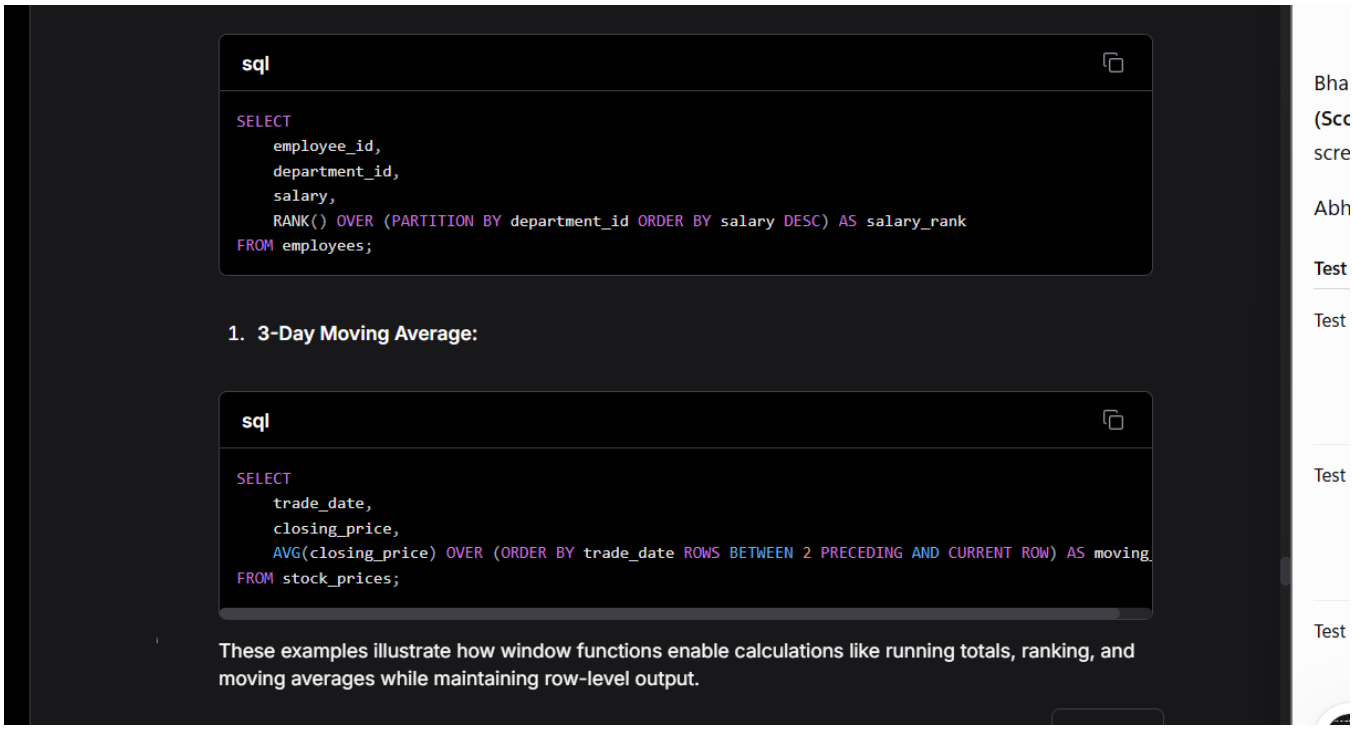
sql

```sql
SELECT
    employee_id,
    department_id,
    salary,
    RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS salary_rank
FROM employees;
```

1. **3-Day Moving Average:**

sql

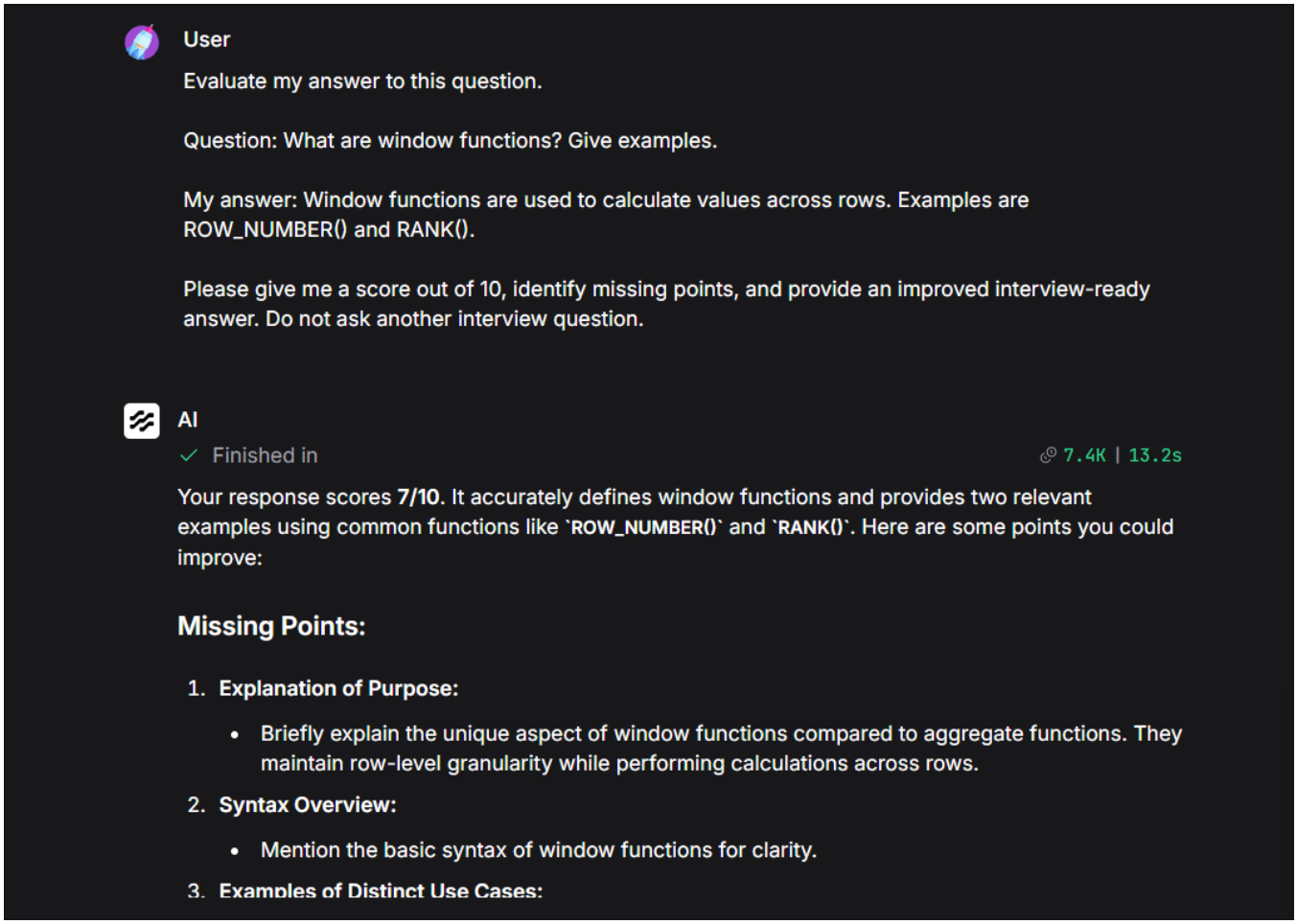
Screenshot 2 — Test 1 — Knowledge Base Retrieval



Screenshot 3 — Test 1 — Knowledge Base Retrieval

Test 2 — Answer Evaluation (Score: 7/10)

Observed result: the user's answer received 7/10. The feedback identifies missing explanation of purpose, syntax overview, and additional use cases such as running totals and moving averages.



Screenshot 1 — Test 2 — Answer Evaluation (Score: 7/10)

3. **Examples of Distinct Use Cases:**
    - Include at least one example that demonstrates a different use case, such as calculating moving averages or running totals.
4. **Specific Functions:**
    - Consider listing more specific window functions, e.g., `AVG()` in a moving average context or `SUM() OVER` with `ROWS BETWEEN`.

Improved Interview-Ready Answer:

Definition and Purpose: Window functions compute values across a set of rows related to the current row without collapsing the result set. They perform calculations over a defined "window" of rows while keeping each row distinct in the output, unlike aggregate functions (e.g., `SUM()`, `COUNT()`) that reduce rows.

Syntax Overview:

sql

```sql
function_name(arguments) OVER (
    [PARTITION BY partition_expression, ...]
    [ORDER BY order_expression [ASC | DESC], ...]
    [ROWS | RANGE between_clause]
)
```

Examples:

Screenshot 2 — Test 2 — Answer Evaluation (Score: 7/10)

1. **Ranking within Groups:**

```sql
SELECT
    employee_id,
    department_id,
    salary,
    RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS salary_rank
FROM employees;
```

This ranks employees by salary within each department.

1. **Calculating a Running Total:**

```sql
SELECT
    order_id,
    amount,
    SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM orders;
```

This calculates a cumulative sum of orders as each date progresses.

1. **Moving Average:**

Screenshot 3 — Test 2 — Answer Evaluation (Score: 7/10)

1. **Moving Average:**

```sql
SELECT
    trade_date,
    closing_price,
    AVG(closing_price) OVER (ORDER BY trade_date
                             ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moving_average
FROM stock_prices;
```

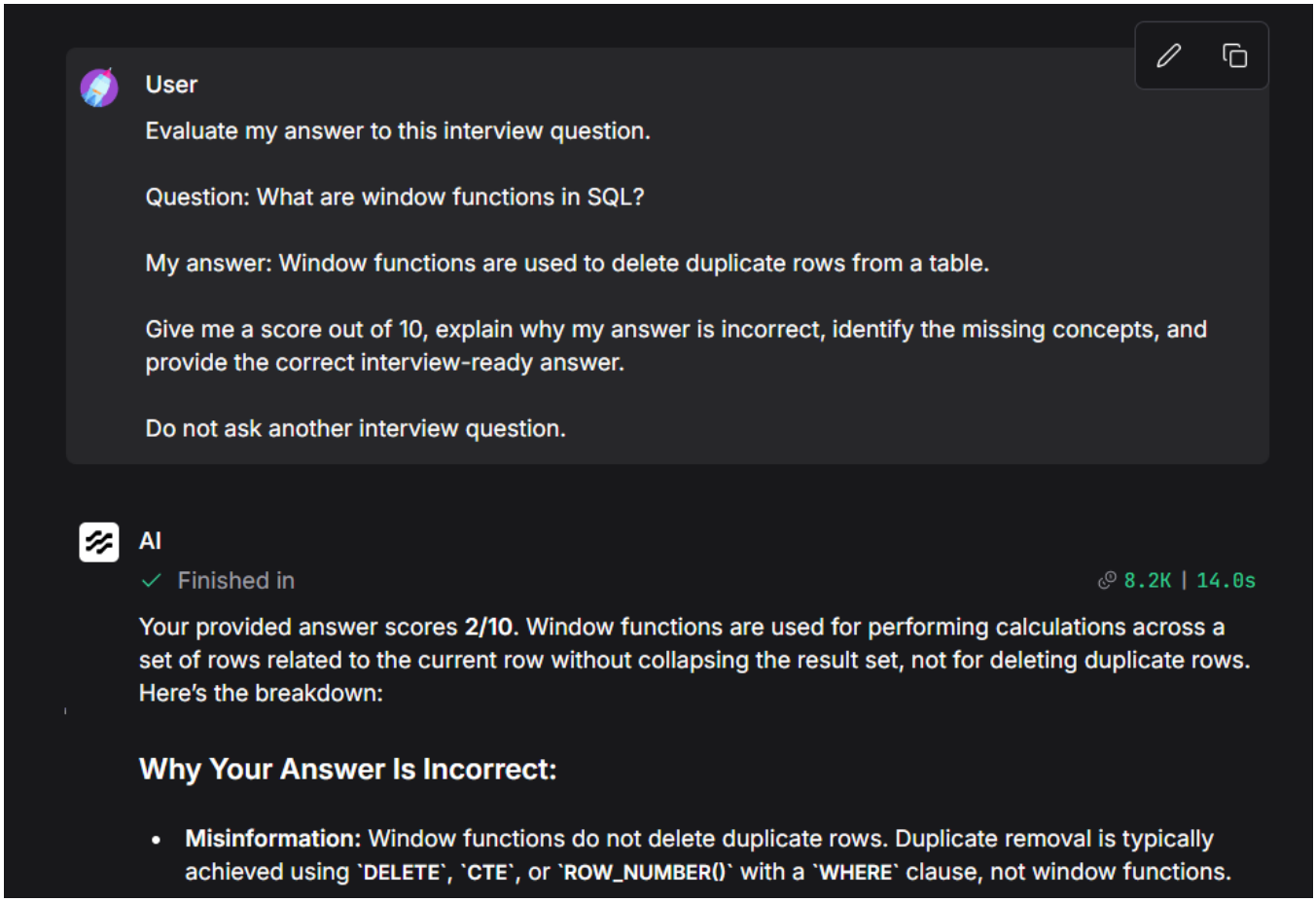
This computes a 3-day moving average of stock prices.

These examples illustrate how window functions facilitate complex analytical queries while preserving row-level details, making them invaluable for data analysis tasks.

Screenshot 4 — Test 2 — Answer Evaluation (Score: 7/10)

Test 3 — Incorrect Answer Evaluation (Score: 2/10)

Observed result: the claim that window functions delete duplicate rows received 2/10. The feedback explains the misconception, lists missing concepts, and provides an improved interview-ready answer with SQL examples.



Screenshot 1 — Test 3 — Incorrect Answer Evaluation (Score: 2/10)

Missing Concepts:

1. **Core Functionality:**
   - Window functions perform calculations across a defined set of rows (the window) while keeping each row in the result set.
2. **Key Concepts:**
   - **PARTITION BY:** Divides rows into partitions to which the window function is applied.
   - **ORDER BY:** Orders rows within each partition for functions like `ROW_NUMBER`, `LAG`, or `LEAD`.
3. **Common Window Functions:**
   - `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`, `LEAD()`, `LAG()`, `SUM() OVER`, `AVG() OVER`, etc.

Correct Interview-Ready Answer:

Definition: Window functions are used to perform calculations across a set of rows related to the current row without collapsing the result set. They help maintain row-level detail while aggregating or transforming data within a specified "window" of rows. Unlike aggregate functions, window functions return one result per row, enabling complex calculations over subsets of data.

Syntax Overview: The syntax for a window function includes the function name followed by `OVER()`, which defines the window:

sql

```
function_name(arguments) OVER (
```

Screenshot 2 — Test 3 — Incorrect Answer Evaluation (Score: 2/10)

sql

```
function_name(arguments) OVER (
    [PARTITION BY partition_expression, ...]
    [ORDER BY order_expression [ASC | DESC], ...]
    [ROWS | RANGE between_clause]
)
```

Examples:

1. **Ranking Rows within Partitions:**

sql

```
SELECT
    employee_id,
    department_id,
    salary,
    RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS salary_rank
FROM employees;
```

This example ranks employees within each department based on their salary.

1. **Calculating Moving Averages:**

sql

Screenshot 3 — Test 3 — Incorrect Answer Evaluation (Score: 2/10)

sql

```sql
SELECT
    trade_date,
    closing_price,
    AVG(closing_price) OVER (ORDER BY trade_date
                             ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS three_day_moving_avg
FROM stock_prices;
```

This calculates a 3-day moving average of stock prices, illustrating the calculation of values over a sliding window of rows.

1. **Identifying Top Performers:**

sql

```sql
SELECT
    *
FROM (
    SELECT
        order_id,
        amount,
        ROW_NUMBER() OVER (PARTITION BY region ORDER BY amount DESC) AS rn
    FROM sales
) t
WHERE rn <= 3;
```

This identifies the top three sales orders by amount within each region.

Screenshot 4 — Test 3 — Incorrect Answer Evaluation (Score: 2/10)

1. **Identifying Top Performers:**

sql

```sql
SELECT
    *
FROM (
    SELECT
        order_id,
        amount,
        ROW_NUMBER() OVER (PARTITION BY region ORDER BY amount DESC) AS rn
    FROM sales
) t
WHERE rn <= 3;
```

This identifies the top three sales orders by amount within each region.

Conclusion: Window functions are powerful tools for analytical queries, allowing calculations such as rankings, moving averages, and running totals while preserving row-level details. They enhance SQL's capability to handle complex analytical tasks efficiently.

Screenshot 5 — Test 3 — Incorrect Answer Evaluation (Score: 2/10)